

FinanceFlow

Personal Finance Management Application

A Modern Full-Stack Web Application

Web Programming Project

Project Documentation & Technical Report

Built with Node.js, Express.js, MySQL/PostgreSQL, and Vanilla
JavaScript

April 2026

Contents

1	Project Overview	2
2	What is Used in the Front-End and Why	2
3	What is Used in the Back-End and Why	4
4	Which Database is Used and Why	5
5	System Architecture	6
6	Database Schema	6
6.1	Users Table	6
6.2	Transactions Table	7
6.3	Goals Table	7
6.4	User Settings Table	7
7	REST API Endpoints	8
8	Key Features	8
8.1	Dashboard	8
8.2	Income & Expense Management	9
8.3	Analytics	9
8.4	Savings Goals	9
8.5	Multi-Currency Support	9
8.6	User Experience	9
9	Project Structure	9
10	Deployment	10
10.1	Platform: Vercel	10
10.2	Database: Neon PostgreSQL	10
10.3	How It Works	11
11	Security Features	11
12	Technology Stack Summary	12

Project Overview

FinanceFlow is a modern, full-stack personal finance management application designed to help users track income, manage expenses, set savings goals, and gain financial insights through interactive analytics.

Key Highlights:

- Real-time dashboard with financial statistics and charts
- Full CRUD operations for income and expense tracking
- Savings goals with visual progress tracking
- Analytics with financial health scoring (0–100)
- Multi-currency support (INR, USD, EUR, GBP, JPY)
- Responsive design with dark/light mode
- Zero-configuration startup with in-memory fallback
- Cloud deployment on Vercel with serverless functions

What is Used in the Front-End and Why

Front-End Technology Stack

Core: Vanilla JavaScript (ES6+)

The frontend is built with **pure HTML5, CSS3, and vanilla JavaScript** — no framework (React, Angular, Vue) is used.

Why Vanilla JavaScript?

- **Zero build step** — no Webpack, Vite, or bundler configuration needed. Files are served directly.
- **No dependency overhead** — no `node_modules` on the frontend; the app loads instantly.
- **Simplicity** — easier to understand, debug, and maintain without framework abstractions.
- **Performance** — smaller bundle size, faster first paint, no virtual DOM overhead.
- **Direct deployment** — static HTML/JS files deploy seamlessly on Vercel as-is.
- Uses modern ES6+ features: `async/await`, arrow functions, template literals, destructuring, modules.

Styling: Tailwind CSS (via CDN)

Why Tailwind CSS?

- **Utility-first approach** — rapid UI development by composing utility classes directly in HTML.
- **Built-in responsive design** — breakpoints (`sm:`, `md:`, `lg:`) for mobile-first

layouts.

- **Dark mode support** — native `darkMode`: "class" configuration for theme toggling.
- **No custom CSS files** — reduces stylesheet maintenance burden.
- **Plugins used**: `forms` (better form styling) and `container-queries` (component-level responsiveness).
- Custom theme with project-specific colors (`#0d968b` primary), border radii, and font family.

Visualizations: Chart.js

Why Chart.js?

- **Simple API** — create bar charts, donut charts, and line charts with minimal code.
- **Interactive** — hover tooltips, click events, and animations out of the box.
- **Responsive** — charts automatically resize to fit containers.
- **Lightweight** — single CDN include, no build integration needed.
- Used for: monthly income vs. expense comparison (bar chart), category breakdown (donut chart), trends.

Icons: Google Material Symbols

- Over 2,500+ icons available via CDN.
- Consistent design language across all pages.
- Variable font with customizable weight, fill, and optical size.

Typography: Google Fonts (Inter)

- Clean, modern sans-serif typeface optimized for screens.
- Weights 300–900 loaded for typographic hierarchy.

Frontend Architecture

The frontend follows a **Multi-Page Application (MPA)** architecture. Each page (dashboard, income, expenses, analytics, settings) is a self-contained HTML file with embedded JavaScript. Shared utilities (`financeflow.js`, `darkmode.js`, `toast.js`) are loaded via script tags. All API communication goes through a centralized **FinanceFlow** API client object using the Fetch API.

What is Used in the Back-End and Why

Back-End Technology Stack

Runtime: Node.js (v18+)

Why Node.js?

- **JavaScript everywhere** — same language on both frontend and backend, reducing context switching.
- **Non-blocking I/O** — handles concurrent database queries and API requests efficiently.
- **npm ecosystem** — access to the largest package registry in the world.
- **Vercel-native** — first-class support for Node.js serverless functions on Vercel.
- **ES Modules** (`"type": "module"`) used throughout for modern `import/export` syntax.

Framework: Express.js v5

Why Express.js?

- **Minimal and flexible** — unopinionated framework that doesn't impose structure.
- **Industry standard** — most widely used Node.js web framework with massive community.
- **Middleware architecture** — composable pipeline for request processing.
- **Easy REST API creation** — simple route definitions for CRUD operations.
- **v5 features** — improved error handling with async route support.

Security Middleware

- **Helmet** — sets secure HTTP headers (X-Frame-Options, CSP, HSTS, etc.).
- **CORS** — Cross-Origin Resource Sharing for frontend-backend communication.
- **express-rate-limit** — prevents API abuse via request rate limiting.
- **compression** — gzip compression for response payloads, reducing bandwidth.

TypeScript (Partial)

- Used for services (`dataService.ts`), routes, and middleware.
- Provides type safety for data models (User, Transaction, Goal, Settings).
- Compiled with `tsc` to JavaScript for production; `ts-node` for development.

Configuration: dotenv

- Loads environment variables from `.env` files.
- Supports database URLs, ports, SSL settings, and feature flags.

Which Database is Used and Why

Database Technology

Primary: MySQL 8 & PostgreSQL (Dual Support)

FinanceFlow supports **both MySQL and PostgreSQL** through an intelligent auto-detection layer. The `connection.mjs` module inspects the `DATABASE_URL` prefix to determine the dialect:

- URLs starting with `postgres://` or `postgresql://` → PostgreSQL mode
- All other URLs or discrete `DB_*` variables → MySQL mode

Why MySQL?

- **Widely used** — one of the most popular relational databases, well-documented.
- **ACID compliant** — ensures data integrity for financial transactions.
- **InnoDB engine** — supports foreign keys, row-level locking, and crash recovery.
- **Easy local setup** — runs on any OS with simple installation.
- **Driver: `mysql2`** with promise support and connection pooling (10 connections).

Why PostgreSQL?

- **Cloud-native** — first-class support on Vercel via **Neon** (managed PostgreSQL).
- **Production-grade** — advanced query optimizer, JSON support, extensibility.
- **Free tier** — Neon offers generous free-tier for hobby projects.
- **Driver: `pg`** with native connection pooling.

Why Dual Database Support?

- **Flexibility** — developers can use MySQL locally and PostgreSQL in cloud production.
- **Vendor independence** — not locked into a single database provider.
- A query translation layer converts MySQL `? placeholders` to PostgreSQL `$1, $2` syntax automatically.

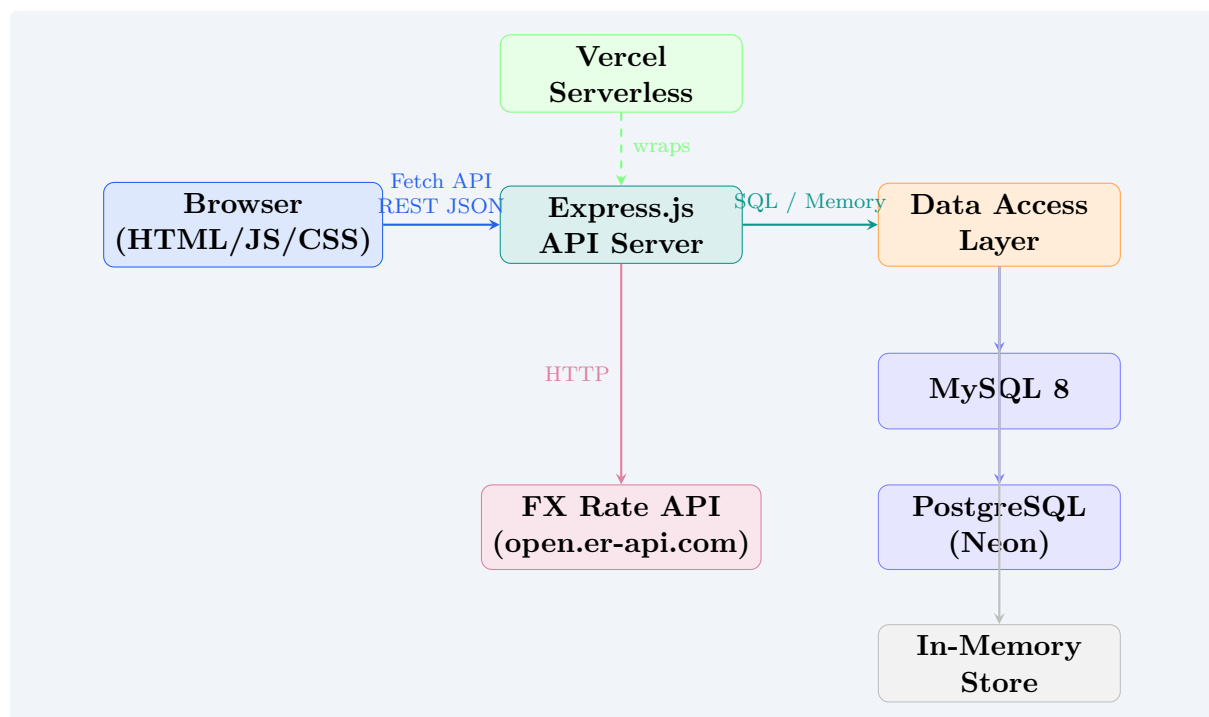
In-Memory Fallback

- When no database is configured, the app uses **JavaScript arrays** as an in-memory store.
- **Zero-configuration startup** — the app works immediately without any database setup.
- Data resets on server restart (ideal for demos and development).

Why No ORM?

- **Direct SQL** with parameterized queries gives full control over query optimization.
- **SQL injection prevention** — all user inputs are passed as bound parameters, never concatenated.
- **Transparency** — every query is visible and debuggable; no hidden ORM magic.
- **Performance** — no ORM overhead or N+1 query issues.

System Architecture



Database Schema

The database consists of **4 core tables** with foreign key relationships:

Users Table

Column	Type	Description
id	CHAR(36) PK	UUID primary key
name	VARCHAR(255)	User's full name
email	VARCHAR(320) UNIQUE	User's email address
created_at	TIMESTAMP	Account creation date

Transactions Table

Column	Type	Description
id	BIGINT PK AUTO_INCREMENT	Transaction ID
user_id	CHAR(36) FK → users	Owner reference
type	ENUM('income', 'expense')	Transaction type
description	VARCHAR(255)	Transaction detail
category	VARCHAR(100)	Category (Salary, Food, etc.)
amount	DECIMAL(12,2)	Amount in specified currency
currency	CHAR(3) DEFAULT 'INR'	Currency code
occurred_at	DATETIME(3)	When the transaction occurred
created_at	TIMESTAMP	Record creation time

Index: `ix_transactions_user_time` (`user_id`, `occurred_at` DESC) for fast user-specific queries.

Goals Table

Column	Type	Description
id	BIGINT PK AUTO_INCREMENT	Goal ID
user_id	CHAR(36) FK → users	Owner reference
name	VARCHAR(255)	Goal name
target_amount	DECIMAL(12,2)	Target savings amount
saved_amount	DECIMAL(12,2) DEFAULT 0	Amount saved so far
currency	CHAR(3) DEFAULT 'INR'	Currency code
created_at	TIMESTAMP	Creation time
updated_at	TIMESTAMP	Last update time

User Settings Table

Column	Type	Description
user_id	CHAR(36) PK FK → users	Owner (1:1 with users)
currency	CHAR(3) DEFAULT 'INR'	Preferred display currency
monthly_goal	DECIMAL(12,2) NULL	Monthly savings target
email_notif	BOOLEAN DEFAULT FALSE	Email notifications
dark_mode	BOOLEAN DEFAULT TRUE	Dark mode preference
two_factor	BOOLEAN DEFAULT FALSE	2FA enabled
updated_at	TIMESTAMP	Last update time

Relationships

All tables use **ON DELETE CASCADE** foreign keys to the `users` table, ensuring referential integrity — when a user is deleted, all their transactions, goals, and settings are automatically removed.

REST API Endpoints

Method	Endpoint	Description
<i>Authentication</i>		
POST	/api/auth/login	User login with email & password
POST	/api/auth/signup	Register a new user account
<i>Dashboard</i>		
GET	/api/dashboard	Full dashboard data (stats, goals, recent txns)
GET	/api/dashboard/stats	Financial statistics only
<i>Transactions</i>		
GET	/api/transactions	List transactions (filter by type, category, search)
GET	/api/transactions/:id	Get a single transaction
POST	/api/transactions	Create a new transaction
PUT	/api/transactions/:id	Update an existing transaction
DELETE	/api/transactions/:id	Delete a transaction
<i>Analytics</i>		
GET	/api/analytics	Analytics overview (monthly, categories)
GET	/api/analytics/overview	Detailed analytics with configurable period
<i>Goals</i>		
GET	/api/goals	List all savings goals
POST	/api/goals	Create a new goal
PUT	/api/goals/:id	Update goal progress
DELETE	/api/goals/:id	Delete a goal
<i>Settings & Utilities</i>		
GET	/api/settings	Get user settings
PUT	/api/settings	Update user settings
GET	/api/fx/latest	Foreign exchange rates (6h cached)
GET	/health	Server health check

Key Features

Dashboard

- Real-time stats: total income, total expenses, balance, savings rate
- **Financial Health Score** (0–100) calculated from:
 - 40% weight: Savings rate
 - 30% weight: Expense consistency
 - 30% weight: Income growth (month-over-month)
- Interactive charts (Chart.js): income vs. expense bar chart, category donut chart
- Recent transactions list with quick actions
- Savings goals with visual progress bars

Income & Expense Management

- Full CRUD operations (Create, Read, Update, Delete)
- Category-based organization:
 - **Income:** Salary, Freelance, Investment, Business, Rental, Other
 - **Expense:** Food, Transport, Shopping, Housing, Entertainment, Healthcare, Education, Other
- Search and filter functionality
- Pagination (10 items per page)
- Monthly totals and trend tracking

Analytics

- Weekly and monthly trend analysis
- Category-wise expense breakdown
- Income vs. expense comparison charts
- Top spending categories ranked
- Financial health insights and recommendations

Savings Goals

- Create goals with target amounts
- Track saved progress with percentage
- Visual progress indicators

Multi-Currency Support

- Supports INR, USD, EUR, GBP, JPY
- Real-time foreign exchange rates from `open.er-api.com`
- 6-hour server-side cache + 1-hour client-side cache for FX rates
- Locale-aware currency formatting via `Intl.NumberFormat`

User Experience

- **Dark/Light mode** toggle with `localStorage` persistence
- **Responsive design** — mobile-first with Tailwind breakpoints
- **Toast notifications** for user feedback
- **Custom modal system** for confirmations and prompts
- **Material icons** for consistent visual language

Project Structure

```
financeflow/  
|-- financeflow/                # Frontend (Vanilla JS +  
    Tailwind)  
|   |-- dashboard/              # Main dashboard page  
|   |-- income_page/            # Income tracking page
```

```
| |-- expenses_page/           # Expense management page
| |-- reports_analytics/       # Analytics with Chart.js
| |-- settings/                # User settings page
| |-- auth/                    # Login & Signup pages
| |-- add_transaction_modal/    # Shared transaction modal
| +-- shared/                  # Shared utilities
|     |-- financeflow.js       # API client library
|     |-- darkmode.js          # Dark mode toggle
|     +-- toast.js             # Toast notifications
|
|-- server/                    # Backend (Node.js + Express)
|   |-- src/
|   |   |-- full-server.mjs    # Main Express server
|   |   |-- data-access.mjs    # Hybrid data layer
|   |   |-- db/
|   |   |   |-- connection.mjs # DB connection (auto-detect)
|   |   |   |-- queries.mjs    # SQL query definitions
|   |   |   +-- migrate.ts     # Migration runner
|   |   |-- middleware/        # Error handler, validators
|   |   |-- routes/            # API route handlers
|   |   +-- services/          # Business logic (TypeScript)
|   +-- migrations/
|       +-- 001_init.sql       # Initial schema (4 tables)
|
|-- site/public/               # Alternative premium UI
|-- api/index.js               # Vercel serverless wrapper
|-- vercel.json                # Vercel deployment config
+-- package.json               # Root dependencies
```

Deployment

Platform: Vercel

- The application is deployed on **Vercel** as a serverless function.
- `api/index.js` wraps the Express app for Vercel's serverless runtime.
- `vercel.json` rewrites all routes to the serverless handler.
- Static frontend files are bundled with the serverless function.

Database: Neon PostgreSQL

- Cloud PostgreSQL via **Neon** for production persistence.
- Connection via `DATABASE_URL` environment variable.
- Auto-migration: schema is applied on first connection.
- SSL enabled for secure cloud connections.

How It Works

1. Browser sends request to Vercel edge
2. Vercel routes to `api/index.js` serverless function
3. Express processes the request through middleware pipeline
4. Data access layer queries Neon PostgreSQL
5. JSON response returned to the browser

Security Features

Feature	Implementation
SQL Injection Prevention	Parameterized queries with bound parameters (<code>?</code> / <code>\$1</code>)
Security Headers	Helmet.js sets X-Frame-Options, CSP, HSTS, X-Content-Type-Options
Rate Limiting	express-rate-limit prevents brute-force and API abuse
Response Compression	gzip compression via compression middleware
Referential Integrity	Foreign keys with ON DELETE CASCADE
Input Validation	Server-side validation on all transaction and settings endpoints
Error Handling	Centralized error middleware; stack traces hidden in production
CORS	Configured Cross-Origin Resource Sharing

Technology Stack Summary

Layer	Technology	Purpose
<i>Frontend</i>		
Language	Vanilla JavaScript (ES6+)	Application logic, DOM manipulation
Styling	Tailwind CSS (CDN)	Utility-first responsive styling
Charts	Chart.js	Interactive financial visualizations
Icons	Material Symbols	Consistent UI iconography
Fonts	Google Fonts (Inter)	Typography
<i>Backend</i>		
Runtime	Node.js 18+	Server-side JavaScript execution
Framework	Express.js v5	REST API routing and middleware
Language	TypeScript (partial)	Type-safe services and routes
Security	Helmet, CORS, Rate Limit	HTTP security and access control
Config	dotenv	Environment variable management
<i>Database</i>		
Primary DB	MySQL 8 / PostgreSQL	Relational data storage
MySQL Driver	mysql2	Connection pooling, promises
PG Driver	pg	PostgreSQL connectivity
Fallback	In-Memory (JS Arrays)	Zero-config development mode
Cloud DB	Neon	Managed PostgreSQL for Vercel
<i>DevOps & Deployment</i>		
Hosting	Vercel	Serverless deployment
Build	TypeScript Compiler	TS → JS compilation
External API	open.er-api.com	Foreign exchange rates